GROUP - 66 Schedular Documentation

Steps to run on terminal:

1. Path of the file
2. Make
3. Make Jobs (I have 5 test jobs in my folder)
4. ./SimpleShell NCPU TSLICE (eg - ./SimpleShell 2 100)
5. Prompt SimpleShell$ will be displayed.
6. Submit ./job1
7. Ctrl-c for termination report

Implementation:

- SimpleShell

  processSubmit() creates process using fork + execvp, but immediately stops it from executing by SIGSTOP (as we will wait for signal SIGCONT from scheduler)

  Shared memory created by main() - Proc Table is shared memory structure for both SimpleScheduler and SimpleShell

  Semaphores used to sync SimpleShell and SimpleSchedular for procTable to avoid race condition

  Launches the scheduler as daemon process

  Ctrl-C (SIGINT for termination of both SimpleShell and SimpleScheduler by sending SIGTERM signal to Scheduler)

  SIGCHLD to detect when job completes and shared memory is updated by handle_sigchld()

- SimpleSchedular

  Runs as daemon process (runs in bg)

  Reads shared memory created by shell

  Schedules and then sleeps for 1 Tslice

Implements RR scheduling policy - picks NCPU jobs from ready queue, runs them for Tslice, preempts (pauses) by SIGSTOP and requeues them if not complete resume NCPU jobs again using SIGCONT

Updates shared memory with process's execution and waiting time using completion_time and waiting_time

- Proper Error Handling implemented in both SimpleShell and SimpleScheduler for system call failures as well as termination. Input errors and Cleanups are also properly handled

Github repository link: https://github.com/antelope0112/Simple-Scheduler